

Large References, Small Checks: Oracle-Guided Porting of a Stochastic Learning System

Anonymous authors

Paper under double-blind review

Abstract

An expensive reference can be reused for small checks throughout the development of a stochastic learning implementation. We contribute a process-level synthesis of heuristic Good–Turing-guided sampling, frozen observed-support sets, and empirical 50%-mass (p50) heads for recurring port checks, applied to Metacat’s single-run and episodic behavior. The central engineering result is its role in guiding a direction-sensitive image repair and resolving a resource-cap mismatch; incomplete historical build provenance limits independent replication. A subsequent versioned study records 969,000 single-run observations, including three reference-engine failures. Six of eight flagged port observations also occur in held-out reference data, exposing finite-reference omissions. A memory-retaining episodic study selects answers by native quality and conceptual preference. Only three of 19 problems meet the coverage target for both populations; five are ineligible for validation. One qualified quality oracle flags 24/100 outside port winners versus 0/1,000 in reference validation, with cause unresolved. Independent validation, not the stopping heuristic, supports coverage claims. Alternative internally justifiable outcome populations might improve construction but remain untested. Documented port improvement is the primary result; we do not establish measured speedup, port equivalence, or improved learning.

1 Introduction

Stochastic implementations are difficult to compare when their output distributions, rather than individual seeded trajectories, express the intended behavior. A language port may consume random draws in a different order, so matching seeds across implementations need not yield matching executions. Nevertheless, reimplementation errors can alter scientifically important outcomes. Researchers need ways to identify such changes without interpreting every disagreement between individual runs as a defect.

We study this problem through a Python reimplementation of Metacat (Marshall, 1999; Marshall & Hofstadter, 1997), an analogy-making architecture extending Copycat (Mitchell, 1993; Hofstadter & Fluid Analogies Research Group, 1995). For an input such as `abc -> abd; xyz -> ?`, probabilistically scheduled computational units, or *codelets*, construct and revise candidate relationships until the program produces an answer or reaches a stopping condition. Different answers can reflect different interpretations of the same analogy. This makes agreement with a particular answer an insufficient general test of the implementation.

The engineering objective is **asymmetric cost over a development lifecycle**: invest once in a versioned reference artifact, then reuse it for smaller samples as the port changes. Our question is operational: **can an expensive, reusable reference make repeated checks of a stochastic learning implementation small and informative?**

The central result is the approach’s role in improving the port. Concrete problem–answer discrepancies guided a direction-image repair and cap reconciliation, connecting small checks to source-level diagnosis and regression testing. Subsequent versioned studies evaluate reference qualification and remaining discrepancies; they complement this development history without retroactively identifying its missing build records.

We report two named-outcome flags. A **NOVEL** flag denotes a check outcome outside the *observed* reference set, not necessarily outside the reference’s true support. A **MISSING** flag denotes a selected high-frequency reference outcome absent from the check, not proof that the port cannot reach it. Both require investigation.

Metacat’s episodic memory makes the learning mode essential to this question. Stored experiences affect later runs, so matching memory-free answers would leave an important part of the reimplementations untested. We therefore examine both memory-free runs and fixed-horizon episodes with memory retained between runs. Testing fidelity of that learning mechanism is distinct from showing that it improves task performance or transfers to unseen analogy problems.

The method combines differential testing, empirical head selection, and missing-mass estimation. These ingredients and asymmetric sample allocation have prior art. We claim **process-level novelty** in their integration and application to oracle-guided port development across single-run and episodic outcomes, not a new estimator or an optimal support-equality test. The contributions are:

1. An integrated workflow combining Good–Turing-guided reference construction, frozen observed-support sets, and empirical p50 heads for small recurring checks, with explicit amortized-cost accounting and reference-reuse conditions.
2. An episode-level formulation for testing memory-dependent learning behavior, alongside ordinary fresh-memory runs, using concrete outcome reports.
3. An analysis separating reference-distribution plug-in baselines from valid statements about support equality, including counterexamples to transferring those baselines under unrestricted probability reweighting.
4. A development case study showing how the workflow helped improve specific port behavior, with a documented repair investigation and audited before/after records, followed by versioned single-run and episodic evaluations of frozen oracles, held-out discoveries, reference failures, and remaining port discrepancies.

The machine-learning question is implementation fidelity when a model’s state depends on experience. This case study evaluates a cognitive architecture’s stochastic reasoning and episodic behavior, not generalization to modern generative models. When a scientific claim concerns outcome probabilities, distributional testing remains necessary alongside named-outcome diagnostics.

2 Background and Related Work

The absence of an immediately available correctness oracle is an established software-testing problem (Weyuker, 1982; Barr et al., 2015; Patel & Hierons, 2018). A second implementation can serve as a pseudo-oracle (Davis & Weyuker, 1981) or differential-testing reference (McKeeman, 1998), but the interpretation of disagreement depends on the reference’s validity and on the semantics being compared. An intentionally stochastic model differs from a test whose nondeterminism is incidental flakiness (Luo et al., 2014).

Metamorphic testing replaces individual expected answers with relations between executions (Chen et al., 1998; 2018; Segura et al., 2016). Statistical metamorphic testing and work on stochastic optimization explicitly address random outputs (Guderlei & Mayer, 2007; Yoo, 2010). Such relations complement the present diagnostic when domain knowledge supports them. Statistical testing of randomized algorithms requires a specified null, effect size, and sampling protocol (Arcuri & Briand, 2014). Statistical model checking instead estimates or tests stated properties of stochastic executions (Younes & Simmons, 2002; Legay et al., 2010); its sequential guarantees do not follow merely from inspecting any statistic repeatedly.

Good–Turing estimation (Good, 1953) estimates probability mass assigned to outcomes not yet observed. Practical frequency estimation (Gale & Sampson, 1995), unseen-vocabulary estimation (Efron & Thisted, 1976), richness estimation (Chao, 1984), and coverage-based estimation (Chao & Lee, 1992) address related but distinct questions. Finite-sample missing-mass analysis supplies confidence-dependent error terms (McAllester & Schapire, 2000), not permission to treat an observed singleton ratio as a certified upper limit. The optimality results of Orlitsky et al. (2016) concern extrapolating unseen species counts, not the diagnostic or stopping rule studied here.

STADS (Böhme, 2018) and residual-risk estimation in greybox fuzzing (Böhme et al., 2021) connect species-discovery statistics with software testing. Capture-recapture methods have also been used for fault-content estimation (Eick et al., 1992). We apply a missing-mass estimate to a reusable reference outcome sample, while recognizing that guarantees developed for particular testing or adaptive-sampling schemes do not automatically apply to ours. In particular, STADS already treats program behaviors, including outputs, as species. The distinction evaluated here is the use of a frozen reference artifact for repeated, named-outcome port checks, not the invention of coverage-guided software testing.

Asymmetric sampling also has direct prior art. Bhattacharya & Valiant (2015) analyze distributional closeness testing with unequal sample sizes and heavy/light outcome separation. Their target is equality versus specified distributional separation, whereas our reports localize observed set differences without such a calibrated verdict. Frequency-sensitive methods can reuse reference samples too. The meaningful comparison is recurring cost and diagnostic behavior under stated targets, not reuse versus an alternative artificially forced to rebuild its reference. The count-ranked cumulative-mass head is an empirical smallest covering region (O’Neill, 2022), not a new selection algorithm; 50% is a design choice, not an established optimal testing threshold.

Alignment and replication of simulation models have long distinguished levels of agreement (Axtell et al., 1996; Wilensky & Rand, 2007). Observed reachability is weaker than distributional equivalence, and matching final answers is weaker than matching internal cognitive processes. Testing probabilistic programming systems (Dutta et al., 2018) is another relevant application area, but this paper does not evaluate the present diagnostic on a probabilistic programming benchmark. Reset-based finite-trace testing of probabilistic stateful systems is established (Gerhold & Stoelinga, 2018). It provides the appropriate comparison for our fixed-episode formulation. Missing-mass estimation for dependent Markov sequences (Pananjady et al., 2024) further shows why ordinary singleton counts cannot simply be pooled across an arbitrarily evolving learning history.

The process-level contribution connects these established tools into a construction-freeze-check-investigate workflow for stochastic port development. We have not identified this complete process in the prior work examined here; this is a bounded comparison, not an exhaustive priority claim. The single-run studies and historical episodic endpoint checks use p50 heads. The subsequent native-best-answer study extends construction and independent validation to two episode-level outcome populations, evaluating frozen supports rather than a new p50 head. This integration and learning-mode application are contributions alongside the primary evidence of port improvement.

3 Asymmetric Cost and Reference Reuse

Let C_R denote the upfront cost of constructing, validating, and storing an empirical reference under a fixed protocol. Let $C_{P,k}$ be the cost of check cycle k , including sampling and comparison, and let $C_{F,k}$ be its follow-up investigation cost. Across K port-check cycles that can legitimately reuse the reference, total and average costs are

$$C_{\text{total}}(K) = C_R + \sum_{k=1}^K (C_{P,k} + C_{F,k}), \quad \overline{C}(K) = \frac{C_R}{K} + \frac{1}{K} \sum_{k=1}^K (C_{P,k} + C_{F,k}).$$

This accounting identity expresses the design objective: tolerate a substantial upfront investment to reduce recurring sampling and interpretation work. It is not itself evidence of a speedup. Reference validation, cap-resolution reruns, storage, and investigation must not disappear from the accounting merely because they occur outside the nominal short check. A baseline using the same reference investment must be allowed to amortize it on the same terms.

For a single-run protocol, the sampling asymmetry is N_x reference runs versus n_x runs per check, usually with $N_x \gg n_x$. For a fixed-horizon learning protocol, those units become whole episodes. An episode can be much more expensive than a memory-free run, especially when memory changes termination behavior. Run and episode counts describe allocation, while measured execution time or compute consumption is needed

to describe actual cost. Neither unit automatically translates into a wall-clock advantage across Scheme, Python, CPU, GPU, or different concurrency settings.

The reusable artifact comprises counts, observed support, selected head, reference implementation and dependency identities, input schedules, initial memory, stopping conditions, seeds, and outcome definitions. Reuse is justified only while these define the same intended reference behavior. Changing an episode horizon, initial memory, cap, or interpretation of an outcome is not merely another port revision: it changes the reference question and requires separate construction or a justified transformation of the artifact. A confirmed reference defect also invalidates affected portions of the reference.

Reference reuse across adaptively developed revisions does not produce independent evidence or a family-wide false-alarm guarantee. Developers may overfit a fixed diagnostic. Independent final audit samples and known-defect controls are therefore important even when the routine oracle remains fixed. The intended contribution is an economical investigation workflow, not a claim that unlimited reuse preserves statistical error control.

4 Diagnostic Construction

4.1 Outcomes, Samples, and Scope

For input x , let $p_R(o \mid x)$ and $p_P(o \mid x)$ denote the reference and port probabilities of outcome o in a countable outcome space $\mathcal{O}$. The true support is $\text{supp}(p) = \{o : p(o) > 0\}$. A reference sample of N_x runs supplies counts $c_x(o)$ and the observed set

$$S_x = \{o : c_x(o) > 0\}, \quad \hat{p}_R(o \mid x) = c_x(o)/N_x.$$

An outcome observed under the specified protocol is evidence of reachability; an unobserved outcome need not be impossible. All probabilities are conditional on a fixed implementation, configuration, input, stopping protocol, and outcome definition. We use n_x for the check size and J for the number of inputs, avoiding reuse of the reference sample size or program symbol.

The historical single-run projection records an answer string, or `*NONE*` for stopping without an answer, or `*CAP*` for exhausting the codelet budget. The versioned continuation in Section 7 additionally records `*ERROR*` for an in-engine failure. Reserved outcomes remain in execution counts and denominators; errors also trigger a separate hard-error flag and are never desired behaviors that the port must reproduce. No errors occur in that study’s construction samples. The projection discards internal structures, their construction order, and timing. It therefore cannot establish process-level fidelity. A larger or infinite outcome space need not prevent a small missing mass, but a highly diffuse distribution may make this projection unhelpful at an affordable budget.

4.2 Learning Episodes as Observations

For episodic learning, the reference condition includes an initial memory M_0 , an input schedule, and a fixed horizon L . Memory persists between runs within an episode and is reset to M_0 before the next episode. Write $\mathcal{E}_{x,e}$ for episode e ’s ordered outcomes and memory evolution, and choose a categorical projection $Z_{x,e} = \phi(\mathcal{E}_{x,e})$ before sampling. Different endpoint or native-evaluation summaries define different projections and therefore different oracles. Our conceptual-outcome checks do not require enumerating or comparing complete episode sequences.

The same set construction applies to the projected observations:

$$c_x^\phi(z) = \sum_{e=1}^{N_x} \mathbf{1}\{Z_{x,e} = z\}, \quad S_x^\phi = \{z : c_x^\phi(z) > 0\}.$$

Here N_x counts whole episodes, not the LN_x dependent runs inside them. Fixed-protocol, independently randomized reset episodes can sample a common episode law, subject to checking for hidden cross-episode state. This is the reset-trace perspective of Gerhold & Stoelinga (2018), not a new independence theorem. Singleton counts and head selection are calculated on projected episodes. The guarantees or caveats for that law do not apply to a pool of all within-episode steps.

The historical learning-mode projection is the last successful answer within eight runs. We call this a *finite-horizon endpoint*, not convergence. A total projection must explicitly retain the case of an episode with no answer. Adding final-run status also distinguishes an answered episode whose last run caps from one that ends with an answer. The archived endpoint comparison and its exclusions are reported in Section 6.5; they are not silently replaced with a newly defined historical reference.

The subsequent study in Section 8 instead uses two projections. **best_a** selects the answer with highest native numerical quality; **best_b** selects a native conceptually preferred answer, undefeated under the native pairwise preference relation. Among equally eligible winning occurrences, the one reached first is selected. Each answered episode supplies one selected answer string to each of two separate frequency tables; the two strings may coincide. Their statistical independence is neither assumed nor required.

The motivation for these populations is conceptual rather than statistical: the episode’s reported result should be internally justifiable by Metacat’s own judgments, rather than chosen solely because it occurred last. Distinct types in each table are distinct selected letter strings. Native evaluation justifies the selection, but does not establish external correctness or retain every structural distinction between answers with the same letters. The choice was not shown to optimize sampling efficiency or coverage estimation.

For these two populations, the denominator counts complete episodes that produce an answer. Entirely answerless and engine-error episodes remain separately recorded, not invented answer types or silently discarded attempts. Coverage statements are therefore conditional on completing with an answer.

A terminal projection can hide changes in learning dynamics. Matching it does not establish matching retrieval, memory updates, or stepwise behavior. Full trajectories and memory-specific controls are necessary to test those claims. Continually growing memory without between-episode resets poses a different dependent, potentially nonstationary problem; stationary Markov missing-mass results (Pananjady et al., 2024) do not automatically resolve it.

4.3 Reference Sampling Is Heuristic

Let $f_{1,x} = |\{o : c_x(o) = 1\}|$. The actual reference missing mass and its Good–Turing estimate are different objects:

$$M_{R,x} = \sum_{o \notin S_x} p_R(o \mid x), \quad \widehat{M}_{R,x} = f_{1,x}/N_x.$$

The estimate is informative about the reference’s unobserved probability mass, not about the fraction of all possible outcomes that has been discovered. It is not an upper confidence bound. In particular, the finite-sample analysis of McAllester & Schapire (2000) concerns independent sampling with confidence-dependent error terms; it does not certify optional stopping on the raw estimate.

The archived single-run sampler inspected merged batch counts and used the following operational rules: after at least 3,000 runs, stop if $f_{1,x} > 0$ and $f_{1,x}/N_x \leq 10^{-4}$; if $f_{1,x} = 0$, require at least 10,000 runs before stopping. A lower reporting threshold of 6×10^{-5} recorded overshooting, not a second acceptance condition. A 50,000-run checkpoint and a 200,000-run hard maximum limited expenditure. Eight workers sampled disjoint interleaved seed indices, while a coordinator merged counts. In-flight work could raise the final count beyond a threshold. Global singleton counts, not independent worker decisions, governed the stopping statistic.

The 10,000-run floor does not certify the target. For example, with two outcomes having probabilities 0.9998 and 0.0002, all 10,000 draws equal the first outcome with probability

$$(0.9998)^{10000} \simeq 0.1353.$$

Then $f_1 = 0$ and the floor is satisfied, yet the missing mass is 2×10^{-4} , twice the target. Also, $f_1 = 1$ at $N = 10,000$ satisfies the positive-singleton rule; the floor is not justified by rejecting that case.

The historical record contains outcome counts and codelet histograms, not the ordered discoveries needed to compare stopping rules. An earlier development account of 35 outcomes and six long discovery gaps could not be reconciled with the archived 10-outcome sample for the same named input and is excluded from the evidence here.

Section 7.4 compares fixed, singleton-based, and no-discovery prefixes using the later versioned study’s ordered observations.

The later episodic rule permits freezing at the first $f_1/N \leq 10^{-4}$, checked after every episode, with no minimum sample floor. Two identical initial winners give $f_1 = 0$ and can therefore freeze a support containing just one answer. This is a *candidate oracle*, not evidence that its missing mass is below 10^{-4} . Section 8 reports the resulting coverage failures without changing the rule after observing them.

4.4 Independent Coverage Qualification

Freeze a candidate support before an independent, fixed-size reference validation batch. Count every validation winner outside the support as a miss, including repetitions of an outside answer. Only a passing one-sided binomial upper confidence bound supports the declared coverage target. A failed or uncollected validation supplies no such guarantee; its observations still remain available for descriptive comparisons. Validation discoveries are never merged into the frozen support to retroactively erase misses.

In the episodic study, both populations must cross the construction threshold before a problem receives validation. The coverage target is 0.01, distinct from the 10^{-4} construction heuristic. A fixed Bonferroni allocation $\alpha = 0.05/38$ covers both definitions for all 19 problems, including those that fail the construction gate. A confidence statement is made only for a passing individual population and is conditional on answered, complete episodes. This does not validate the empirical p50 head, certify port correctness, or transfer the bound to a different port distribution.

Validation belongs in the upfront reference budget. For a frozen S_x , zero misses in m independent reference draws give a one-sided $(1 - \alpha)$ upper limit $1 - \alpha^{1/m}$ on missing mass. At $\alpha = 0.05$, a limit of 10^{-4} requires 29,956 draws per input, before simultaneous confidence allocation. This analytical budget is distinct from the episodic study’s 0.01 target; Section 7.2 reports nominal and simultaneous bounds for the versioned single-run study’s 30,000 validation runs per input.

4.5 Head Selection and Flags

Sort observed outcomes by decreasing count, breaking ties lexicographically by outcome key. Let H_x be the shortest prefix whose cumulative count is at least $N_x/2$. We call this the empirical *p50 head*. This defines a coverage-based selection rule, not a lower bound on every selected outcome’s probability. For a uniform distribution over B outcomes, a member’s probability is $1/B$; even a head member can be arbitrarily rare as B increases. Selection is also uncertain: the reference outcome `copy5/cc` lies exactly at the empirical half-mass boundary, so resampling could change the head size.

Freeze S_x and H_x before checking the port. For the check’s observed set T_x , report

$$\text{MISSING}_x = H_x \setminus T_x, \quad \text{NOVEL}_x = T_x \setminus S_x.$$

MISSING prioritizes a selected reference outcome that was not reproduced; **NOVEL** prioritizes an outcome not previously observed in the reference. Neither is a verdict of correctness. No flag means only that all selected head outcomes were seen and all check outcomes belonged to the observed reference set.

The check compares membership, but frequency determines head selection and the optional baseline calculations. A flag report should retain its input, outcome, count, seeds, configuration, reference sample size, stopping reason, and missing-mass estimate. Novel outcomes are not automatically added to the reference: follow-up may reveal an incomplete reference, a reference defect, a port defect, or a protocol mismatch.

For each input x :

- Collect reference counts under the recorded budget/stopping rule.
- Freeze S_x = observed reference keys.
- Freeze H_x = the count-ranked prefix covering at least half the sample.
- Record counts, singleton ratio, stopping reason, and configuration.
- Record whether independent held-out validation qualifies the support.
- Failed or absent validation supplies no coverage guarantee.
- Never add validation discoveries to the frozen support.

For each check cycle and input x :

- Collect n_x independent-unit port outcomes under a compatible protocol.
- The unit is a fresh-memory run or a whole reset learning episode.
- Report H_x minus observed keys as MISSING.
- Report observed keys minus S_x as NOVEL.
- Attach evidence and optional reference-distribution plug-in baselines.
- Investigate flags; do not infer support equality from a quiet cycle.

5 Statistical Interpretation

5.1 Absence Under the Check Distribution

Conditional on the frozen reference sample, and assuming independent check observations from a stationary distribution, the absence probability for $o \in H_x$ is

$$\Pr(o \notin T_x \mid S_x, H_x) = (1 - p_P(o \mid x))^{n_x}.$$

Substitution of $\hat{p}_R(o \mid x)$ gives a *plug-in reference-distribution baseline*, not a calibrated probability under the null of equal supports. Even under equality of distributions, the plug-in quantity does not account for uncertainty in the reference shares, selection of the head, or reference stopping. Absence of a still-reachable outcome is a false-positive discrepancy flag under a matched-distribution null; it is not the false-negative probability for detecting deletion of that outcome. If $p_P(o \mid x) = 0$, it is absent with certainty in any nonempty check sample.

Counterexample under equal supports. A head outcome with reference share approximately 0.1947 may have port probability 0.001 while both true supports remain identical. Its absence probability in 100 port runs is $0.999^{100} \simeq 0.9048$, not approximately 4×10^{-10} .

If one could justify a lower bound $p_P(o \mid x) \geq \ell_{x,o} > 0$, then $(1 - \ell_{x,o})^{n_x}$ would be an absence upper bound. A union bound over all selected members gives a cycle bound by summing those quantities, without requiring independence between member-absence events. Justifying the lower bounds, including any estimation uncertainty, is separate work. Support equality alone supplies no such positive quantitative lower bound.

5.2 Novelty and Its Counting Units

For a frozen S_x , define $q_{P,x} = \sum_{o \notin S_x} p_P(o \mid x)$. For independent check draws, the expected number D_x of out-of-set draws and the probability of flagging the input are

$$\mathbb{E}[D_x \mid S_x] = n_x q_{P,x}, \quad \Pr(D_x > 0 \mid S_x) = 1 - (1 - q_{P,x})^{n_x}.$$

The harness instead lists *distinct* novel problem–outcome pairs. If U_x is their count, then

$$\mathbb{E}[U_x \mid S_x] = \sum_{o \notin S_x} [1 - (1 - p_P(o \mid x))^{n_x}] \leq n_x q_{P,x}.$$

These are different counting units. Under $p_P = p_R$, $q_{P,x} = M_{R,x}$, which can be estimated by the reference singleton ratio. If only the supports match, however, arbitrarily much port mass can be assigned to reference outcomes not yet observed. The reference missing-mass estimate then need not approximate the check’s novelty probability. Likewise, even an exactly known reference missing mass would not transfer without an assumption controlling probability drift. For example, a justified total-variation bound of d_x would imply $q_{P,x} \leq M_{R,x} + d_x$; no such bound has been established for this port.

If erroneous outcomes have total port mass ϵ and all lie outside S_x , the probability of missing all of them in n_x independent runs is $(1 - \epsilon)^{n_x}$. At $n_x = 100$, detection probabilities for $\epsilon = 10^{-4}, 10^{-3}, 10^{-2}$ are approximately 0.0100, 0.0952, and 0.6340. Short checks can therefore overlook rare defects even when observed head outcomes are easy to reproduce.

5.3 A Frequency-Sensitive Comparator

Membership diagnostics are not substitutes for frequency-sensitive tests, nor are such tests inherently expensive or uninterpretable. Consider a known Bernoulli reference law with $p_R(A) = 0.85$ and a port with $p_P(A) = 0.20$; the complementary outcome has positive probability in both. The reference p50 head is $\{A\}$. A 100-run membership check misses A with probability only $0.8^{100} \simeq 2.04 \times 10^{-10}$, despite the substantial rate change.

For comparison, the explicitly specified binomial test rejecting the known null $p(A) = 0.85$ when $C_A \leq 50$ in 100 draws has null rejection probability

$$\sum_{k=0}^{50} \binom{100}{k} (0.85)^k (0.15)^{100-k} \simeq 2.30 \times 10^{-16},$$

and power approximately 0.999999999995 at $p(A) = 0.20$. These are exact toy-distribution calculations, not a comparison on the Metacat benchmark. Estimating an unknown reference law would require accounting for that estimate. The example establishes the narrower point: low check cost is not unique to membership diagnostics, and a change the latter usually ignores may be easy for a frequency-sensitive test to detect.

6 Development History and Port Improvement

Testing was part of building the Python port, not only an assessment after translation. Short samples against the reference yielded named discrepancies that could be traced through source code, probed, and checked after repair. This development history is the core engineering result: the image-direction investigation links a flag to a semantic correction, while cap reconciliation improves the comparison protocol by identifying a false alarm.

6.1 Systems and Protocol

The Python port was developed through model-assisted translation and repeated comparison with the Scheme implementation and dissertation. Runtime checking complemented source inspection; neither was sufficient alone. The benchmark contains 19 letter-string inputs drawn from demonstrations and simpler analogy cases. Appendix F lists all inputs and selected shared outcomes. Single-run checks start with cleared episodic memory. Learning-mode checks instead retain memory through eight-run episodes and clear it between episodes (Section 6.5). Independence and stationarity of pseudorandomly indexed sampling units remain modeling assumptions, not consequences of clearing memory alone.

The historical reference sampler used a locally modified, headless Metacat 1.2 with a 100,000-codelet cap. The source changes address unsupported object messages, whole-string rule handling, dispatch, and display-free execution. Some changes can affect outcomes; the reference is not represented as unmodified Metacat or as distributionally equivalent to it.

A patch-only reconstruction bundle now makes the modified source available without redistributing the original tarball. However, that current source snapshot does not prove which exact build generated the archived reference sample. The records lack sufficient historical build attribution for an independent end-to-end replication. We report their counts as archived measurements and distinguish the new arithmetic audit from new engine runs.

The saved single-run checks use seeds 900,000–900,099 for each input. Reusing this block before and after repairs creates a paired replay comparison, not independent cycles. Different random-number generators also mean that equal numerical seeds would not pair trajectories between Scheme and Python.

6.2 Reference Summary

The archived reference contains 374,500 runs, 366 observed problem–outcome pairs, and 27 selected head members. A repeated answer on different inputs is counted separately. Reference sizes range from 10,250 to 51,000, averaging 19,710.5 per input. Six inputs have no singletons; three have estimated missing mass above 10^{-4} . Table 1 reports the per-input figures.

Table 1: Archived reference counts. Stopping labels describe the heuristic, not certified completeness.

Input	N_x	$ S_x $	$f_{1,x}$	$f_{1,x}/N_x$	Stored reason	$ H_x $
misc4	11,000	4	0	0.00000000	no_singletons	2
fig5.7	18,125	18	1	0.00005517	saturated	2
misc3	19,000	54	4	0.00021053	shards_exited	2
misc5	11,125	8	1	0.00008989	saturated	3
misc2	11,000	6	1	0.00009091	saturated	1
run1	11,000	11	0	0.00000000	no_singletons	1
run4	11,000	10	1	0.00009091	saturated	2
misc1	30,750	18	3	0.00009756	saturated	1
fig5.4-top	51,000	61	19	0.00037255	checkpoint	2
eqe-baaab	51,000	51	13	0.00025490	checkpoint	1
run6	47,500	80	4	0.00008421	saturated	1
run3	21,000	12	2	0.00009524	saturated	2
run2	11,000	10	1	0.00009091	saturated	1
copy1	10,875	2	0	0.00000000	no_singletons	1
copy2	11,250	1	0	0.00000000	no_singletons	1
copy3	11,000	1	0	0.00000000	no_singletons	1
copy4	11,125	2	0	0.00000000	no_singletons	1
copy5	15,500	13	1	0.00006452	saturated	1
copy6	10,250	4	1	0.00009756	saturated	1

The stored **saturated** label means the *heuristic* threshold was met, not that the true support was enumerated or the true missing mass bounded. The two checkpoint cases end at 51,000 after in-flight work. The remaining above-target case, **misc3**, has stored reason **shards_exited**; development notes describe a manual stop, but the aggregate file does not by itself establish why workers exited. Heterogeneous sample sizes motivate adaptive allocation, but do not demonstrate that this allocation outperforms a fixed-budget alternative.

For $n_x = 100$, the smallest selected empirical share is $4089/21000 = 0.194714\dots$ for **run3/wyz**. Its plug-in absence probability is 3.9355×10^{-10} . Summing the 27 member-specific plug-in absence probabilities yields 6.1340×10^{-10} . This is the cycle union-bound calculation *after substitution of empirical reference rates*, not a confidence guarantee for the real port.

Using the actual singleton ratios produces three distinct plug-in summaries: 0.169485 expected novel draws per cycle, 0.167898 expected inputs with at least one novelty, and probability 0.155914 of any novelty if check samples are also independent across inputs. The last calculation is $1 - \prod_x (1 - \widehat{M}_{R,x})^{100}$. The expected distinct-pair count is bounded by the expected draw count under the corresponding plug-in model, not generally equal to it. Zero-singleton inputs contribute zero to these plug-in sums, not certified zero risk.

The design quantity $19 \times 100 \times 10^{-4} = 0.19$ is not an established bound: three recorded estimates exceed the target, and none is an upper confidence limit. The actual plug-in sum happens to be below 0.19.

6.3 Cap Compatibility and Cost

Each check cycle schedules 100 initial runs per input, or 1,900 in total. The reference-to-initial-check run-count ratio is $374500/1900 = 197.105\dots$. This allocation is subject to the reuse conditions and cost distinctions in Section 3.

The port first runs at a 20,000-codelet working cap. In the repaired single-run harness, capped seeds are rerun at the reference’s 100,000-codelet cap. The saved post-repair cycle records 23 such reruns, so it entails 1,923 execution attempts, not exactly 1,900. The reported 197-fold ratio excludes these extra attempts and the different work per attempt; it is not a measured speedup.

Cap reconciliation is valid as trajectory extension only if seeded replay is deterministic and the larger cap does not alter transitions before the original cutoff. Fresh sessions alone do not guarantee this. Differences in codelet semantics or numerical backends can remain even with equal numeric caps. The case study does not establish general CPU/GPU equivalence.

6.4 Flags and Investigation

Table 2 summarizes the three archived single-run checks. Each contained every selected reference head member. This establishes observed reachability in those check samples, not comparable outcome rates, support equality, or fidelity over all development cycles.

Table 2: Observed flags, not estimated error rates. All cycles reuse the same seed block.

Archived cycle	Novel pairs	Missing head members
CPU before repairs	5	0
CPU after repairs	1	0
Archived MLX check	5	0

The pre-repair CPU cycle reported five novel pairs: `ege-baaab/abbbb`, `run6/cdddb`, `copy1/*NONE*`, `run1/*CAP*`, and `copy5/aac`. Development traces associated the first three with one direction-sensitive image defect (RC-A). A leftward group’s image was constructed in physical left-to-right order with rightward direction. Rendering could conceal the mismatch because reversing both direction and constituent order leaves an untouched string unchanged; subsequent direction-sensitive rule operations exposed it.

The development record contains a Python trace and Scheme-side probes. Injecting the image defect into the reference reportedly produced `cdddb` and `*NONE*`, matching two of the flagged outcomes. The quoted intervention table does not show reproduction of `abbbb`; that finding is supported by the Python trace and its disappearance after repair, not an assertion that every flagged string was reproduced in Scheme. Directional image regression tests provide additional local evidence. The historical instrumented runs have not been independently repeated in this revision.

The `run1/*CAP*` finding (RC-B) arose from comparing different resource caps and disappeared after cap reconciliation. Following the image and harness repairs, the archived CPU replay contained only `copy5/aac`, whose proposed cause was investigated and rejected (RC-C). It remains unresolved. The archived MLX cycle still contains five novel pairs and does not demonstrate that those repairs were validated on that backend.

The development result is therefore not simply that five flags became one: the workflow guided a semantic repair and targeted regression checks, distinguished a protocol mismatch from a port defect, and retained an unresolved finding. These support improvement of specific port behavior, not global correctness, a quantified defect-rate reduction, or superiority over another testing workflow.

A cross-cycle recurrence marker helps prioritize investigation, but supplies no proof of invalidity. A legitimate reference outcome omitted from S_x can recur indefinitely. In these saved cycles, reuse of the same seed block is an additional reason not to interpret recurrence as independent confirmation.

6.5 Episodic Learning-Mode Checks

Learning-mode comparison applies the same asymmetric design to repeated memory-bearing sessions. The historical report describes a reference of 500 episodes per input, or 9,500 episodes across the 19 inputs, each consisting of eight runs on the same problem. The port checks use 100 episodes per input: 1,900 episodes and 15,200 within-episode runs per cycle. One memory is retained inside each episode and a fresh memory is used for the next. The reference therefore represents 76,000 within-episode runs and a 5:1 reference-to-check episode-count ratio. This is a smaller asymmetry than the approximately 197:1 single-run ratio; the latter must not be presented as a learning-mode result. Both reference artifacts are intended for reuse across check cycles.

The 500-episode reference budget should not be confused with satisfying the single-run sampling heuristic in Section 4.3. Stored episodic singleton ratios range up to 0.016, and the episode reference has no independent missing-mass validation in the available record.

Memory mechanism. The port retains answer and snag descriptions across runs while reinitializing the workspace, concept network, scheduler, and per-run reminding activations. Its direct cross-run search feedback is duplicate-answer rejection: the problem strings, answer string, and structural signatures of both the inferred rule and its target translation determine whether an answer has already been stored. The answer finder, justification, and justification-clamp recovery paths consult this memory. A matching answer is rejected, allowing search to continue; the same answer letters obtained by a different rule are not necessarily duplicates. Retaining memory can therefore change both later outcomes and whether a run terminates (Marshall, 1999; Marshall & Hofstadter, 1997). An endpoint over answer letters necessarily loses some of the information that drives this mechanism, so even matching complete letter sequences would not alone verify rule-level memory fidelity.

Reminding and comparison constitute a second, distinct behavior. After an answer is stored, the inspected reporting path compares it with past answers using themes, rules, justification, and coherence, then records graded activations and emits reminding commentary. Stored snag descriptions can also change the retrospective explanation of an otherwise unjustified theme. These explanatory functions are part of the implemented architecture, but are not evidence that retrieval improves subsequent answer quality. Here, learning-mode testing concerns experience-dependent behavior, principally the cross-run duplicate guard, rather than parameter fitting or a demonstrated learning curve. The repeated-problem protocol measures memory-conditioned exploration; it does not by itself measure transfer to new problems or an externally assessed gain in answer quality.

Implementation-level checks. Existing regression definitions address several memory-specific repairs: distinguishing answers reached by different rules; preventing justification-clamp recovery from accepting a remembered answer; comparing snag rules structurally instead of by potentially colliding English descriptions; and giving a past answer zero reminding activation at the distance threshold. Other checks cover the distance components, snag-based reclassification of themes, clearing activations without deleting stored answers, and session-scoped identifiers. The episode harness also requires both an answer-found status and a newly added memory entry before counting a run as successful; an answer left in memory by an earlier run is insufficient. These inspected checks document intended local contracts, not a freshly executed regression result or Scheme/Python trajectory comparison. The archived endpoint changes below cannot be attributed to each of these repairs: their historical build attribution is incomplete.

The saved check files contain all ordered eight-state sequences. We recompute the last successful answer by searching backward past `*NONE*` and `*CAP*`, then verify the saved endpoint histogram and missing-head list. Four episodes per cycle never answer; the legacy histogram omits them but records their count separately. We retain that count in Table 3, rather than representing the success-only histogram as a complete episode distribution. The historical episodic reference itself is not present in the arithmetic supplement, so its 9,500-episode size is attributed to the development record, and stored novelty lists cannot be independently reconstructed here. The count of each listed novel endpoint is verifiable from the port trajectories.

Table 3: Archived eight-run episodes. Novelty labels come from stored flags; the historical episodic reference is unavailable for independent re-computation.

Metric	CPU before	CPU after	Archived MLX
Check episodes	1,900	1,900	1,900
Within-episode runs	15,200	15,200	15,200
Stored novel endpoint pairs	17	13	17
Episodes with stored novel endpoint	28	14	28
Missing stored head members	0	0	0
Episodes never answering	4	4	4
Capped within-episode runs	1,311	2,404	1,309
Episodes with any capped run	628	1,052	626
Episodes ending in a capped run	361	591	361

The paired CPU records contain 17 stored novel input/endpoint pairs across 28 episodes before repair and 13 pairs across 14 episodes afterward. No selected head member is missing in either cycle. Among the post-repair pairs, 11 also appear in the separate single-run reference; the remaining two are `misc2/ajd` and `ege-baaab/qrrbq`, each in one episode. This cross-mode classification is useful context, not evidence that an outcome is correct for a particular memory history. An outcome reachable without memory need not have the same probability, or even be reachable under a chosen learned context.

The historical investigation connects the direction-image repair to the disappearance of `abbbb`, `baaba`, `cdddb`, and `cddbc` from the episodic novelty list. That record complements the single-run findings, but the exact instrumented builds have the provenance limits described in Section 6.1. The archived MLX sequence data are a separate cycle, not validation of a repaired GPU learning implementation.

Two limitations prevent interpreting this as a matched-protocol learning validation. First, the port uses 20,000 codelets per run while the reported reference uses 100,000. A cap early in an episode can alter memory and all subsequent runs. Similar aggregate cap frequencies would not establish that the mismatch is harmless. Moreover, CPU capped runs increase from 1,311 to 2,404 in the saved before/after records, even while novel endpoints decrease. The report’s later prose gives 2,406 capped runs and 1,054 affected episodes; the saved post-repair sequences give 2,404 and 1,052. We use the identifiable saved artifact rather than infer an undocumented intermediate cycle.

Second, episode e , step i uses seed $900000 + 8e + i$, with $0 \leq e < 100$ and $0 \leq i < 8$. Before/after cycles are paired, and these seeds overlap the single-run check block. They are not independent replications across modes.

These limitations motivated the versioned, matched-cap study with native best-answer projections in Section 8. Memory-specific controls remain necessary to isolate the behavior detected by a small check (Section 9).

7 Versioned Single-Run Evaluation

The `support-v1a` campaign supplies versioned single-run measurements with identified builds, ordered observations, frozen construction samples, and fresh held-out checks. It evaluates one port revision, complementing rather than replaying the historical repair sequence.

7.1 Protocol and Post-Failure Amendment

All 19 inputs use fresh memory on every run and a direct 100,000-codelet cap in both implementations. Per input, the design collects 20,000 reference construction runs, 30,000 held-out reference validation runs, and 1,000 port runs: 380,000, 570,000, and 19,000 observations, respectively. Validation and port samples are divided into prespecified 100-run checks, giving 300 reference and ten port checks per input. Each phase uses a disjoint seed block, with seed equal to its recorded phase base plus $100000j + i$ for problem index j and run index i . Independence under a fixed law remains a pseudorandom sampling assumption. Reference execution uses Chez Scheme 9.5.4 under Linux/amd64 emulation; the serial port uses Python 3.14.6 and NumPy 2.5.1. Manifests identify source, parameter data, dependencies, protocol, and receipts.

The original `support-v1` collection halted on a reproducible reference error during validation. A separately versioned, explicitly post-failure amendment retains the original budgets, inputs, seeds, engines, caps, and frozen construction sets. It records in-engine failures as `*ERROR*`, restarts the process, and continues with the next assigned seed; infrastructure and unclassified failures still stop collection. Verified complete parent chunks are inherited once. The interrupted chunk is replayed with its original seeds, its 238 successful prefix observations checked for agreement, and one observation admitted per assigned seed. Incomplete attempts remain available as provenance, not extra independent data. The 570-run pilot, corrected 375-execution preflight, and aborted wrapper-development preflight are excluded from the 969,000 main observations. No engine repair is part of the continuation. This is not an unchanged prospective experiment.

7.2 Frozen-Reference Checks

Table 4 uses the full frozen 20,000-run reference for each input and its empirical p50 head. The new reference has 27 selected head members across the benchmark. All 3,876 main chunks are complete and checksummed. Appendix C gives per-input results and the alternative construction-prefix comparisons.

Table 4: Versioned fresh-memory checks against 20,000 frozen construction runs per input. Counts pool 19 inputs for accounting, not to estimate a common outcome law. Errors are included in outside draws and also reported separately.

Measure	Reference validation	Port
Observations	570,000	19,000
100-run checks	5,700	190
Outside draws (including errors)	124	8
Checks with outside draws	119	6
Checks missing a p50 member	0	0
Engine-error observations	3	0
Frequency rejections at 0.05/19	4	31

Sixteen inputs have held-out discoveries: 124 validation draws lie outside construction, including the three engine errors, in 119 of 5,700 checks. For `misc4`, `misc5`, and `copy6`, zero discoveries in 30,000 validation runs give a nominal one-sided 95% per-input upper bound of approximately 9.9853×10^{-5} . None meets 10^{-4} with simultaneous 19-input coverage: even a zero-discovery input has adjusted upper bound approximately 1.9799×10^{-4} . These bounds concern reference execution outcomes, not a guarantee on the port or a pooled cross-input probability.

The port produces eight outside draws in six of 190 checks. Five distinct input–outcome pairs account for them (Table 5). Six of the eight draws have counterparts in held-out reference data, demonstrating that finite construction can omit reachable reference behavior. The held-out discoveries are not merged into construction to erase the original flags. The two remaining observations, `misc1/*NONE*` and `copy5/acc`, warrant investigation; their absence from these reference samples does not prove a port defect. No check misses a selected p50 member, which establishes sampled reachability of the head, not distributional equivalence.

Frequency-sensitive comparison. On the same frozen references and 100-observation checks, a Monte Carlo permutation comparator uses empirical total-variation distance. Conditional on pooled category

Table 5: All five flagged port input–outcome pairs. Held-out matches do not enlarge the frozen reference or remove recorded flags.

Input	Outcome	Port draws	Held-out reference draws
misc1	*NONE*	1	0
fig5.4-top	qqiqq	1	5
run6	pqqbc	1	2
copy5	acc	1	0
copy5	cbb	4	2

counts, it draws 999 random allocations to the check sample by a multivariate hypergeometric law and reports $(1 + b)/1000$, where b is the number of simulated distances at least as large as observed. It rejects at 0.05/19, yielding four reference-validation and 31 port rejections. The adjustment is across inputs within a check, not across every repeated batch; the minimum attainable reported p-value is 0.001. Reuse of construction samples also makes check results dependent. The comparator tests distribution equality under exchangeability assumptions, whereas membership and head flags localize particular outcomes. These complementary results neither establish calibrated defect-detection power nor show that either procedure dominates the other.

7.3 Reference Failures as Development Evidence

The expensive held-out reference campaign itself exposed three failed executions on allowed inputs:

Input	Seed	Codelets	Exception signature
misc1	20713988	2,835	Non-procedure #f
misc1	20716342	1,981	Non-procedure #f
misc3	20226148	2,088	caddr: incorrect list structure #f

These are three executions with two exception signatures, not three independently diagnosed bugs. The first failing continuation identifies a missing letter-category descriptor in `make-group`; the shared or distinct root causes of the other failures remain unresolved. Errors stay in all execution denominators and also trigger hard-error flags, even if their category were present in a reference. A port is not required to reproduce reference defects. There are no port engine errors in this campaign.

This result extends the development lesson: building and qualifying the oracle is also a stress test of the reference, and retained failures provide concrete targets for regression and repair. Ordinary exception detection here is not attributed specifically to Good–Turing or p50. No post-failure engine patch was introduced to remove inconvenient observations from the study.

7.4 Construction Choices and Amortized Cost

The ordered 20,000-run construction samples permit comparison of fixed prefixes of 1,000, 5,000, 10,000, and 20,000 runs per input with two heuristics. At 500-run checkpoints after at least 3,000 observations, the singleton rule accepts $f_1/N \leq 10^{-4}$, with a 10,000-run floor when $f_1 = 0$; the no-discovery rule requires a gap of at least 1,000 runs. A rule that never fires retains its 20,000-run prefix. The singleton rule fires on 14 inputs and uses 268,500 construction observations including five truncated inputs; it leaves 135 held-out outside draws and nine port outside draws. The no-discovery rule fires on all 19 and uses 81,500 observations, leaving 346 and 34 outside draws, respectively. Table 11 reports all six choices. These share observations and have unequal construction budgets; they are not independent, equal-cost experiments or evidence of an optimal estimator.

Construction plus validation costs 950,000 reference executions, compared with 19,000 port executions across ten 100-run checks per input: a 50:1 aggregate count ratio. One benchmark-wide check uses 1,900 port runs,

so that upfront reference investment is 500 times one check’s execution count. The ten checks sample one frozen port, not ten successive repaired revisions. They instantiate the asymmetric allocation without measuring lifecycle savings. Both the membership diagnostic and frequency comparator reuse the reference on the same terms. Recorded aggregate attempt wall time is approximately 466,573 seconds, including inherited, interrupted, continuation, and corrected preflight attempts. It is neither campaign elapsed time nor CPU time; pilot, earlier diagnosis, wrapper-development preflight, setup, human effort, and analysis costs are not fully included.

8 Episodic Frozen-Oracle Coverage

The versioned episodic v3 study uses all 19 problems, eight runs per episode, and a 100,000-codelet per-run cap for both implementations. Memory is retained within an episode and reset between episodes. It collects the separate `best_a` and `best_b` answer-string populations defined in Section 4.2, not sequences or combinations of co-winners. This study evaluates their frozen supports; it does not add a new p50-head experiment.

Construction and validation. Each population freezes independently at its first eligible $f_1/N \leq 10^{-4}$ crossing. Construction ends as soon as both have frozen or after 2,000 episodes per problem. Four exploratory pilot problems contribute their full previously collected 1,000-episode prefixes; their initial freeze eligibility is evaluated at the whole prefix, not retrospectively at an earlier crossing. The other 15 problems can freeze immediately. This asymmetric pilot reuse is explicit, not an unchanged pre-pilot sampling design. New episode first seeds are sampled independently with replacement; fresh validation is separate from construction.

A problem whose two supports froze receives exactly 1,000 reference validation episodes. Problems reaching the construction cap without both crossings receive none. Every problem still receives 100 port episodes, with descriptive comparisons retained even when reference coverage is unqualified. All assigned episodes and failures are retained. No construction extension, extra validation batch, or retrospective oracle enlargement was performed.

Table 7 reports successful, failed, and skipped validation. Bounds use the fixed 38-population confidence allocation in Section 4.4. Five `copy5` validation episodes were entirely answerless, giving an answered denominator of 995; the other tested populations have 1,000. The main campaign has no new engine-error episodes; its one construction error is inherited from the pilot. The study records 10,074 construction, 14,000 validation, and 1,900 port episodes: 25,974 episodes and 207,791 actual inner runs, including inherited pilot executions. Preflight and serializer-diagnostic executions are separate overhead, not scientific samples.

Only `copy1`, `copy2`, and `copy3` meet the coverage target for both populations. Eight of the 28 tested individual populations pass: those six, `misc3 best_a`, and `copy4 best_b`. Twenty tested populations fail; ten populations on five capped problems are not validated. Each passing population has zero misses among 1,000 answered validation episodes, giving an adjusted upper bound of approximately 0.006611. A passing population does not confer coverage on its partner.

Failure of early freezing. `misc5` froze both supports after two episodes, yet subsequent validation produced 480/1,000 `best_a` and 591/1,000 `best_b` winners outside them. Immediate zero-singleton stopping thus frequently produced incomplete supports, which independent validation exposed. The lower construction budget does not establish learning-induced sampling savings.

Population choice. The native definitions were chosen to make the selected episode results and their distinct answer types internally justifiable, not because their discovery frequencies were known to suit Good–Turing-guided construction. Good–Turing may have worked more effectively with a different population of episodic results; a future experiment could choose another conceptually motivated projection to build a better oracle. That requires new construction and independent validation, not transfer of existing bounds. These data neither isolate population choice from immediate stopping nor show that an alternative is better. A changed projection also cannot turn a raw zero estimate into a confidence certificate (Section 4.3). The current failed checks remain results, not grounds for redefining the study.

Table 7: Episodic frozen-oracle coverage. E counts construction episodes; N_v counts answered validation episodes. A and B denote the native quality and conceptual-preference populations. Pass lists definitions meeting the 0.01 coverage target with 95% simultaneous confidence across all 38 populations. Dashes denote an unmet freeze or uncollected validation, not zero misses.

Input	Build E	Freeze A	Freeze B	N_v	Miss A	Miss B	Pass
<code>misc4</code>	2,000	—	1,000	—	—	—	not tested
<code>fig5.7</code>	8	8	6	1,000	37	111	none
<code>misc3</code>	11	11	7	1,000	0	99	A
<code>misc5</code>	2	2	2	1,000	480	591	none
<code>misc2</code>	2	2	2	1,000	6	63	none
<code>run1</code>	2,000	1,000	—	—	—	—	not tested
<code>run4</code>	2,000	—	1,000	—	—	—	not tested
<code>misc1</code>	14	5	14	1,000	589	193	none
<code>fig5.4-top</code>	2,000	—	—	—	—	—	not tested
<code>eqe-baaab</code>	2,000	2	—	—	—	—	not tested
<code>run6</code>	21	21	2	1,000	101	425	none
<code>run3</code>	2	2	2	1,000	377	418	none
<code>run2</code>	2	2	2	1,000	251	242	none
<code>copy1</code>	2	2	2	1,000	0	0	A,B
<code>copy2</code>	2	2	2	1,000	0	0	A,B
<code>copy3</code>	2	2	2	1,000	0	0	A,B
<code>copy4</code>	2	2	2	1,000	2	0	B
<code>copy5</code>	2	2	2	995	36	29	none
<code>copy6</code>	2	2	2	1,000	3	2	none

All-problem port checks. Table 8 reports both populations for every input, including zero-outside checks and problems without validated coverage. All problems have 100 assigned port episodes; `copy5` has four entirely answerless episodes, so both of its winner denominators are 96. The other port denominators are 100. A number in an outside column describes membership in an independently frozen support, not a defect verdict. A dash means that population never froze; a support that froze while its partner did not can still have descriptive membership counts, but receives no coverage validation under the problem-level gate.

Of the eight coverage-qualified populations, only `misc3 best_a` has outside port winners: 24/100. The other seven have zero outside winners, including both populations of `copy1`, `copy2`, and `copy3`, and `copy4 best_b`. Those zero counts do not establish equal frequencies, port equivalence, or correct internal memory behavior. Large outside counts also occur against unqualified supports: `misc5` has 47/100 and 62/100, and `run3` has 45/100 and 54/100, for A and B respectively. These remain descriptive findings; failed reference qualification prevents interpreting them using the requested reference-coverage guarantee.

For the five problems reaching the 2,000-episode construction limit, Appendix D reports descriptive frequency comparisons for both populations. These use all collected construction episodes, not a frozen prefix or validation data. For example, `misc4 best_a` selects `b` once among 2,000 reference episodes but 20 times among 100 port episodes. This supplies a concrete investigation target even without qualified reference coverage. No new p-values, equivalence thresholds, or retrospective oracle enlargement are introduced.

Termination accounting. The port attempts all 15,200 scheduled inner runs: 9,815 answer, 3,388 terminate without an answer below the cap, and 1,997 reach the codelet cap. There are no port engine errors. Caps occur in 863 episodes; an episode with capped or answerless inner runs can still supply a winner from another run. The four entirely answerless port episodes are retained as execution outcomes but supply neither A nor B. Appendix D gives disjoint inner-run counts for every phase and documents the partially executed inherited error episode. The winner comparisons are conditional on answered complete episodes, not an assessment of equal answer-production or termination probabilities.

Table 8: Port results for all 19 inputs, each assigned 100 eight-run episodes. N_p counts answered complete episodes; No answer counts entirely answerless episodes; Cap runs counts inner runs reaching the codelet limit (out of 800). Outside counts refer to independently frozen supports. A dash means no frozen support, not zero outside winners. Ref. pass repeats the independently validated reference populations; it is not a port pass/fail verdict.

Input	N_p	No answer	Cap runs	Outside A	Outside B	Ref. pass
misc4	100	0	0	–	0	not tested
fig5.7	100	0	0	2	13	none
misc3	100	0	66	24	26	A
misc5	100	0	0	47	62	none
misc2	100	0	406	0	8	none
run1	100	0	122	0	–	not tested
run4	100	0	0	–	0	not tested
misc1	100	0	97	24	36	none
fig5.4-top	100	0	0	–	–	not tested
ege-baaab	100	0	0	6	–	not tested
run6	100	0	0	7	33	none
run3	100	0	0	45	54	none
run2	100	0	189	23	28	none
copy1	100	0	233	0	0	A,B
copy2	100	0	215	0	0	A,B
copy3	100	0	224	0	0	A,B
copy4	100	0	229	0	0	B
copy5	96	4	0	2	3	none
copy6	100	0	216	0	0	none

8.1 A Qualified Oracle Exposes a Port Discrepancy

The misc3 analogy is `abc -> aabbcc`; `kkjjii -> ?`: the example changes single letters into pairs, while the target consists of three descending same-letter pairs. Both frozen supports contain `kji`, `kkjjii`, and `kkkjjjiii`, which have one, two, and three occurrences of each target letter, respectively. These structural descriptions are not unique correctness labels. `best_a` freezes after 11 episodes with counts 3, 6, and 2; `best_b` freezes after seven with counts 2, 3, and 2. Their freeze statistics are separate even though their observed supports coincide.

Held-out reference validation selects those three strings as `best_a` in 466, 319, and 215 of 1,000 episodes. Zero outside winners give the adjusted upper missing-mass bound 0.006611, meeting the 0.01 target. In contrast, `best_b` selects 99 outside winners, with upper bound 0.130501, failing that target. The 100 port episodes select 24 outside `best_a` winners across ten strings, and 26 outside `best_b` winners across 16 strings. Every outside port string occurs among the reference validation’s selected `best_b` answers (Appendix B). The discrepancy therefore concerns episode-winner distributions, not demonstrated impossibility of generating those strings. The failed `best_b` qualification permits only a descriptive port comparison; it does not invalidate the separately qualified `best_a` reference.

The 24 outside quality-winner episodes partition into 21 with a single maximum-quality string, one with a tie among outside strings, and two with an outside/in-oracle tie. Only the latter two memberships could change by selecting another tied winner. The predeclared earliest-occurrence policy is retained. Nine outside episodes answered on all eight runs, so truncation cannot explain the entire discrepancy. The reference’s episode-maximum quality has median 94, compared with 86 for the port; 988/1,000 reference maxima exceed 90, versus 0/100 port maxima. These are descriptive diagnostic findings, not a new calibrated hypothesis test or a causal partition of bugs.

Static inspection identifies two differences in the port’s rule-generalization path: exclusion of literal common-change schemas that the reference retains, and omission of eligibility checks when generalizing changes to an object’s components. The saved winner descriptions do not contain the rule clauses or workspace states

needed to attribute particular events to either difference. Appendix B details these leads and the evidence limits. Neither engine was repaired and no observation was removed or reclassified.

For this problem, 11 construction plus 1,000 validation episodes represent 8,088 reference runs, compared with 800 runs in the 100-episode port check. This makes the smaller check informative through concrete outside winners, without establishing a measured speedup or a memory-specific cause.

9 Limits and Further Evaluation

Calibration and power. The interpretation in Section 5 is diagnostic, not a calibrated test of unrestricted support equality. The versioned studies supply held-out reference checks, but neither their flag counts nor frequency-test rejections estimate detection power against a controlled population of defects. Known defect injections and equal-budget reusable-reference baselines would test what small checks detect and whether the allocation rule improves cost or power. The current unequal-budget prefix comparisons and illustrative binomial example do not establish either advantage.

Historical evidence. The repair history remains the core engineering result, subject to the sampled-build and intervention limits in Section 6.1 and Section 6.4. Appendix A distinguishes reproducible saved-data analyses from historical evidence that the supplements cannot recover.

Selection and scope. Inputs and reference/check parameters were selected during development, not preregistered. The benchmark is small and belongs to one architecture. The learning-mode outcome comparisons do not establish faithful memory trajectories, beneficial learning, or transfer to new problems. Continuous outputs, high-dimensional outputs, and adaptive generative pipelines are not evaluated. Scheduler, precision, or concurrency changes cannot be assumed to preserve support merely because they were intended as legitimate engineering changes. Outcome-level agreement can also conceal different internal processes.

Learning-specific controls. Memory-retained versus memory-cleared episodes and known memory-defect controls are needed to isolate what experience-dependent behavior a small check detects. The native-best-answer study tests memory-bearing episodes but does not diagnose a memory defect or separate population choice from stopping behavior. Alternative projections remain the untested direction described in Section 8.

Lifecycle evaluation. Neither the historical record nor aggregate execution accounting measures complete development costs across revisions, including investigation and human effort (Section 3). Prospective repair cycles with fresh check blocks would assess recurrence without replaying the same seeds and measure actual reference reuse. A comparison with development without this workflow is also needed to quantify its incremental benefit.

10 Conclusion

Large references made small checks useful during Metacat port development: named discrepancies guided a direction-sensitive image repair, exposed a cap-mismatch false alarm, and preserved unresolved findings for investigation. That documented improvement is the core engineering result, despite incomplete historical provenance. The subsequent versioned studies strengthen its evaluation rather than recreate the repairs: single-run checks reveal finite-reference omissions, complementary frequency differences, and reference failures; episodic checks test native quality and conceptual-preference winners with memory retained.

The episodic results also show why heuristic construction must be separated from independent coverage qualification. Only three of 19 problems qualify both populations, yet one qualified oracle identifies a concrete discrepancy: 24/100 outside `misc3` quality winners in the port versus 0/1,000 in reference validation. Its cause remains unresolved. Alternative internally justifiable outcome populations might improve future construction, but require new evidence.

The process-level contribution integrates Good–Turing guidance, reusable observed-support oracles, and p50 diagnostics, extending the workflow to episodic outcomes. Novelty lies in the integration and application, not a new estimator. Lifecycle savings and learning-specific sensitivity still require prospective cost comparisons and memory controls. Quiet checks do not prove equivalence, and outside answers do not prove defects.

References

- Andrea Arcuri and Lionel Briand. A hitchhiker’s guide to statistical tests for assessing randomized algorithms in software engineering. *Software Testing, Verification and Reliability*, 24(3):219–250, 2014. doi: 10.1002/stvr.1486. URL <https://doi.org/10.1002/stvr.1486>.
- Robert Axtell, Robert Axelrod, Joshua M. Epstein, and Michael D. Cohen. Aligning simulation models: A case study and results. *Computational and Mathematical Organization Theory*, 1(2):123–141, 1996. doi: 10.1007/BF01299065. URL <https://doi.org/10.1007/BF01299065>.
- Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015. doi: 10.1109/TSE.2014.2372785. URL <https://doi.org/10.1109/TSE.2014.2372785>.
- Bhaswar Bhattacharya and Gregory Valiant. Testing closeness with unequal sized samples. In *Advances in Neural Information Processing Systems*, volume 28, 2015. URL <https://proceedings.neurips.cc/paper/2015/hash/5cce8dede893813f879b873962fb669f-Abstract.html>.
- Marcel Böhme. STADS: Software testing as species discovery. *ACM Transactions on Software Engineering and Methodology*, 27(2):7:1–7:52, 2018. doi: 10.1145/3210309. URL <https://doi.org/10.1145/3210309>.
- Marcel Böhme, Danushka Liyanage, and Valentin Wüstholtz. Estimating residual risk in greybox fuzzing. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pp. 230–241, 2021. doi: 10.1145/3468264.3468570. URL <https://doi.org/10.1145/3468264.3468570>.
- Anne Chao. Nonparametric estimation of the number of classes in a population. *Scandinavian Journal of Statistics*, 11(4):265–270, 1984. URL <https://www.jstor.org/stable/4615964>.
- Anne Chao and Shen-Ming Lee. Estimating the number of classes via sample coverage. *Journal of the American Statistical Association*, 87(417):210–217, 1992. doi: 10.1080/01621459.1992.10475194. URL <https://doi.org/10.1080/01621459.1992.10475194>.
- Tsong Yueh Chen, S. C. Cheung, and S. M. Yiu. Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, Department of Computer Science, Hong Kong University of Science and Technology, 1998. URL <https://arxiv.org/abs/2002.12543>. Reissued on arXiv in 2020.
- Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51(1):4:1–4:27, 2018. doi: 10.1145/3143561. URL <https://doi.org/10.1145/3143561>.
- Martin D. Davis and Elaine J. Weyuker. Pseudo-oracles for non-testable programs. In *Proceedings of the ACM ’81 Conference*, pp. 254–257, 1981. doi: 10.1145/800175.809889. URL <https://doi.org/10.1145/800175.809889>.
- Saikat Dutta, Owolabi Legunsen, Zixin Huang, and Sasa Misailovic. Testing probabilistic programming systems. In *Proceedings of the 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pp. 574–586, 2018. doi: 10.1145/3236024.3236057. URL <https://doi.org/10.1145/3236024.3236057>.
- Bradley Efron and Ronald Thisted. Estimating the number of unseen species: How many words did Shakespeare know? *Biometrika*, 63(3):435–447, 1976. doi: 10.1093/biomet/63.3.435. URL <https://doi.org/10.1093/biomet/63.3.435>.
- Stephen G. Eick, Catherine R. Loader, M. David Long, Lawrence G. Votta, and Scott Vander Wiel. Estimating software fault content before coding. In *Proceedings of the 14th International Conference on Software Engineering (ICSE)*, pp. 59–65, 1992. doi: 10.1145/143062.143090. URL <https://doi.org/10.1145/143062.143090>.

- William A. Gale and Geoffrey Sampson. Good–Turing frequency estimation without tears. *Journal of Quantitative Linguistics*, 2(3):217–237, 1995. doi: 10.1080/09296179508590051. URL <https://doi.org/10.1080/09296179508590051>.
- Marcus Gerhold and Mariëlle Stoelinga. Model-based testing of probabilistic systems. *Formal Aspects of Computing*, 30:77–106, 2018. doi: 10.1007/s00165-017-0440-4. URL <https://doi.org/10.1007/s00165-017-0440-4>.
- I. J. Good. The population frequencies of species and the estimation of population parameters. *Biometrika*, 40(3–4):237–264, 1953. doi: 10.1093/biomet/40.3-4.237. URL <https://doi.org/10.1093/biomet/40.3-4.237>.
- Ralph Guderlei and Johannes Mayer. Statistical metamorphic testing: Testing programs with random output by means of statistical hypothesis tests and metamorphic testing. In *Seventh International Conference on Quality Software (QSIC)*, pp. 404–409, 2007. doi: 10.1109/QSIC.2007.4385527. URL <https://doi.org/10.1109/QSIC.2007.4385527>.
- Douglas R. Hofstadter and Fluid Analogies Research Group. *Fluid Concepts and Creative Analogies: Computer Models of the Fundamental Mechanisms of Thought*. Basic Books, 1995. ISBN 9780465051540.
- Axel Legay, Benoît Delahaye, and Saddek Bensalem. Statistical model checking: An overview. In *Runtime Verification (RV)*, volume 6418 of *Lecture Notes in Computer Science*, pp. 122–135. Springer, 2010. doi: 10.1007/978-3-642-16612-9_11. URL https://doi.org/10.1007/978-3-642-16612-9_11.
- Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pp. 643–653, 2014. doi: 10.1145/2635868.2635920. URL <https://doi.org/10.1145/2635868.2635920>.
- James B. Marshall. *Metacat: A Self-Watching Cognitive Architecture for Analogy-Making and High-Level Perception*. PhD thesis, Indiana University, Bloomington, 1999. URL <https://science.slc.edu/jmarshall/metacat/dissertation.pdf>.
- James B. Marshall and Douglas R. Hofstadter. The Metacat project: A self-watching model of analogy-making. *Cognitive Studies: Bulletin of the Japanese Cognitive Science Society*, 4(4):57–71, 1997. doi: 10.11225/jcss.4.4_57. URL https://doi.org/10.11225/jcss.4.4_57.
- David A. McAllester and Robert E. Schapire. On the convergence rate of Good–Turing estimators. In *Proceedings of the 13th Annual Conference on Computational Learning Theory (COLT)*, pp. 1–6, 2000. URL <https://www.learningtheory.org/colt2000/papers/McAllesterSchapire.pdf>.
- William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998. URL https://www.bitsavers.org/pdf/dec/dtj/dtj_v10-01_1998.pdf.
- Melanie Mitchell. *Analogy-Making as Perception: A Computer Model*. MIT Press, 1993. URL <https://mitpress.mit.edu/9780262132893/analogy-making-as-perception/>.
- Ben O’Neill. Smallest covering regions and highest density regions for discrete distributions. *Computational Statistics*, 37:1229–1254, 2022. doi: 10.1007/s00180-021-01172-6. URL <https://doi.org/10.1007/s00180-021-01172-6>.
- Alon Orlitsky, Ananda Theertha Suresh, and Yihong Wu. Optimal prediction of the number of unseen species. *Proceedings of the National Academy of Sciences*, 113(47):13283–13288, 2016. doi: 10.1073/pnas.1607774113. URL <https://doi.org/10.1073/pnas.1607774113>.
- Ashwin Pananjady, Vidya Muthukumar, and Andrew Thangaraj. Just wing it: Near-optimal estimation of missing mass in a Markovian sequence. *Journal of Machine Learning Research*, 25(312):1–43, 2024. URL <https://jmlr.org/papers/v25/24-0511.html>.

- Krishna Patel and Robert M. Hierons. A mapping study on testing non-testable systems. *Software Quality Journal*, 26(4):1373–1413, 2018. doi: 10.1007/s11219-017-9392-4. URL <https://doi.org/10.1007/s11219-017-9392-4>.
- Sergio Segura, Gordon Fraser, Ana B. Sanchez, and Antonio Ruiz-Cortés. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42(9):805–824, 2016. URL <https://eprints.whiterose.ac.uk/id/eprint/110335/>.
- Elaine J. Weyuker. On testing non-testable programs. *The Computer Journal*, 25(4):465–470, 1982. doi: 10.1093/comjnl/25.4.465. URL <https://doi.org/10.1093/comjnl/25.4.465>.
- Uri Wilensky and William Rand. Making models match: Replicating an agent-based model. *Journal of Artificial Societies and Social Simulation*, 10(4), 2007. URL <https://www.jasss.org/10/4/2.html>. Article 2.
- Shin Yoo. Metamorphic testing of stochastic optimisation. In *Third International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, pp. 192–201, 2010. doi: 10.1109/ICSTW.2010.26. URL <https://doi.org/10.1109/ICSTW.2010.26>.
- Håkan L. S. Younes and Reid G. Simmons. Probabilistic verification of discrete event systems using acceptance sampling. In *Computer Aided Verification (CAV)*, volume 2404 of *Lecture Notes in Computer Science*, pp. 223–235. Springer, 2002. doi: 10.1007/3-540-45657-0_17. URL https://doi.org/10.1007/3-540-45657-0_17.

A Reproduction and Evidence Boundaries

The standalone LaTeX ZIP builds the manuscript. A separate experimental review supplement contains the historical measurements, both new studies’ scientific exports, frozen port source and seed data, analysis code, tests, and Metacat reconstruction patches. Its root README maps each paper result to its evidence and provides dependency links. The package manifest inventories included bytes, original source hashes, documentation replacements, and exclusions. No original upstream Metacat source or runtime binary is redistributed; the reconstruction helper downloads the linked upstream archive, applies the diffs, and verifies the resulting files.

Saved-data verification. From the extracted supplement’s root, `python3 verify.py` checks package and frozen-source hashes, regenerates five audit reports and all thirteen generated data tables, runs the academic and reconstruction-helper tests, and reproduces the episodic study’s stopping decisions, gates, bounds, and frequencies. These steps use Python’s standard library and execute neither engine. The study verifier’s Unix locking import requires macOS, Linux, or a Linux environment such as WSL on Windows.

The historical audits reconstruct the empirical p50 head, stored flags, probability calculations, endpoint counts, and cap accounting from the archived measurement records. They do not recover missing discovery order, identify all historical sampled builds, or regenerate historical episodic novelty from an unavailable reference. These limitations remain despite the development history’s practical role in improving the port.

The episodic all-input audit checks both selected-answer populations and their independently frozen supports against saved episodes. Its report retains all 38 construction/port frequency vectors. The separate `misc3` audit provides complete frequencies, the 24 outside quality-winner events, native winner descriptions, and diagnostic partitions. These calculations do not reconstruct unrecorded native rule clauses, workspace states, or causal interventions. The public episodic export includes observations and selected/co-winning answer descriptions, not the private full attempt archive. The earlier raw-record inspection reported in Appendix B is therefore a documented audit result, not independently reproduced from this public export alone.

The four single-run scientific archives preserve ordered observations, raw attempt evidence, receipts, manifests, the interrupted parent, and excluded pilot/preflight records. Provenance copies must not be concatenated as new observations. The lightweight table audit checks saved batch summaries; full reanalysis is available with the recorded numerical dependencies:

```
python3.14 -m venv .venv-review  
.venv-review/bin/python -m pip install -r studies/support-v1/requirements.txt  
.venv-review/bin/python verify.py --full-single-run
```

This verifies the archived file inventories and inheritance, reruns the frequency permutations and remaining analysis on saved observations, and checks byte-identical analysis outputs. It does not repeat the 969,000 engine runs. NumPy 2.5.1 and SciPy 1.18.1 are pinned; numerical-library changes may alter exact reproduction. Optional reference reconstruction likewise executes no analogy episodes. GUI startup and fresh collection are separate activities, not prerequisites for verifying the reported results.

Review instructions replace two repository README files; repository ignore configuration, a GUI screenshot, private operational metadata, and Git history are omitted. Included scientific records and frozen executable source remain unchanged, as do technical implementation labels and hash chains. Third-party attribution, the Metacat license, and the port’s existing MIT terms are preserved; the latter’s copyright holder display name is anonymized for review. The supplement is not a complete UI/database deployment or a preconfigured campaign relaunch, and it does not repair the historical provenance gaps. No oracle is enlarged, no observation is synthesized, and no original campaign is modified by packaging.

B Episodic Discrepancy Details

Table 9 gives the complete `misc3` answer frequencies. Construction columns refer to independent freeze points, not the final common construction horizon. Validation discoveries are not merged into either frozen support. Counts refer to one selected string per answered episode, not all co-winner occurrences or sequences of answers.

Table 9: Complete `misc3` selected-answer frequencies. Freeze A and B use 11 and seven construction episodes, respectively; reference validation uses 1,000 episodes and the port check uses 100. Only `kji`, `kkjjii`, and `kkkjjjiii` belong to the frozen supports.

Answer	Freeze A	Freeze B	Ref. A	Ref. B	Port A	Port B
jii	0	0	0	1	0	0
kji	3	2	466	380	25	28
kjii	0	0	0	1	0	1
kjiii	0	0	0	8	5	2
kjji	0	0	0	3	0	1
kjjiii	0	0	0	1	1	1
kjjji	0	0	0	9	2	1
kjjjiii	0	0	0	7	4	1
kkji	0	0	0	2	0	0
kkjiii	0	0	0	1	0	0
kkjji	0	0	0	3	0	0
kkjjii	6	3	319	293	15	19
kkjjiii	0	0	0	4	0	0
kkjjiiii	0	0	0	2	0	0
kkjjji	0	0	0	5	0	1
kkjjjii	0	0	0	3	0	0
kkjjjiii	0	0	0	3	0	1
kkjjjjiii	0	0	0	3	0	1
kkkji	0	0	0	9	2	4
kkkjiii	0	0	0	5	3	1
kkkjji	0	0	0	3	1	0
kkkjiii	0	0	0	2	0	1
kkkjjjii	0	0	0	10	4	7
kkkjjjiii	0	0	0	8	1	1
kkkjjjjii	0	0	0	4	0	1
kkkjjjjiii	2	2	215	228	36	27
kkkkjjjii	0	0	0	2	1	1

Selection evidence. All 24 outside port `best_a` descriptions are also conceptually undefeated, marked coherent, and have zero unjustified themes and three themes. Their quality scores range from 84 to 88. Unequal letter multiplicities do not themselves imply native incoherence. In 20 of these episodes, retained descriptions also establish the presence of an in-oracle answer: 18 below the maximum quality, and two tied at the maximum. The other four lack saved inside-answer evidence, which is not proof that no inside answer was generated; losing descriptions were not exhaustively exported.

The exhaustive 21/1/2 diagnostic partition in Section 8.1 concerns winning strings. In one-based port episode 43, `kjjjiii`, `kjjji`, and `kkkjji` tie at quality 86, all outside. In episode 74, selected `kjjji` ties with in-oracle `kkkjjjiii` at 87. In episode 84, selected `kjjjiii` ties with outside `kkkjjjiii` and in-oracle `kkkjjjjiii` at 86. The port selects by original memory order, not the canonical order of exported descriptions. The canonical records do not independently recover arrival order in those three cross-string ties, but changing tie order could not remove the other 22 outside memberships.

Scores and rule-generalization leads. Both inspected answer-quality paths use $\text{round}(0.6Q_{\text{rule}} + 0.4(100 - T))$, where rule quality combines uniformity, abstractness, and succinctness. Matching this outer formula

does not establish matching inputs or search dynamics. For example, at abstractness 96 and uniformity 100, a one-clause rule has succinctness 100 and quality 98; three unit-cost clauses have succinctness 67 and quality 84. At temperature 10, the answer scores are 95 and 86. This is an illustrative calculation, not a reconstruction of any saved rule. Outside strings are not uniformly over-scored relative to reference occurrences: reference preference co-winners with string `kkkji` reach quality 88, while its two selected port quality winners score 86. The shortage of high-scoring port episode maxima is therefore an investigative lead, not evidence that all uneven strings are mechanically invalid.

Two source differences are relevant candidates, without event-level attribution:

1. The port’s common-change filter requires a non-null relation. Metacat’s filter excludes only Identity, retaining a common literal destination without a named relation. A letter-to-group object-category change is an example relevant to interpreting the three changes `a -> aa`, `b -> bb`, and `c -> cc` together. Excluding such schemas can change the available concise component-level rules.
2. The port generalizes common schemas to subobjects without the reference’s checks for component coverage, shared enclosing objects, and descriptors on uncovered components. This can propose generalizations the reference would not propose from that cluster. A subsequent rule evaluator still exists, so the missing checks alone do not prove an invalid proposal survives.

The inspected source hashes match the frozen study manifest and reconstructed reference manifest. A read-only archive audit verified all 1,111 `misc3` completion receipts and their 9,699 named files against the public episode and selection records. The 1,011 reference raw exports agree with the published co-winner descriptions and saved earliest selections. This checks record consistency, not the correctness of all native computations. The archive lacks rule clauses, rule-quality components, workspace groupings, and a complete record of memory interventions. It cannot assign the outside events to proven generation, scoring, translation, or memory root causes. No repair, new episode, or retrospective oracle enlargement is part of this investigation; assessing a repaired port would require a separately versioned comparison. There is also no matched memory-disabled control establishing that learning caused the observed difference.

C Single-Run Evaluation Details

Table 10 reports all 19 inputs for the versioned fresh-memory study. Construction remains frozen; engine errors are retained in the validation and port denominators. Outside-draw counts and hard-error counts overlap, rather than defining disjoint categories. The three validation inputs with no discoveries meet the nominal per-input 10^{-4} coverage target, but none meets it with the 19-input confidence allocation.

Table 10: Versioned single-run results. States counts outcomes in 20,000 construction runs. Reference validation uses 30,000 runs per input and the port 1,000; errors are included in new draws. Flag counts port 100-run checks with an outside draw or absent p50 member, and Freq. counts frequency rejections at 0.05/19, both out of ten checks per input.

Input	States	Ref. new	Ref. err.	Port new	Port err.	Flag	Freq.
misc4	4	0	0	0	0	0	3
fig5.7	20	2	0	0	0	0	0
misc3	58	15	1	0	0	0	8
misc5	8	0	0	0	0	0	0
misc2	9	1	0	0	0	0	0
run1	11	1	0	0	0	0	0
run4	12	1	0	0	0	0	7
misc1	17	4	2	1	0	1	0
fig5.4-top	41	28	0	1	0	1	0
ege-baaab	41	17	0	0	0	0	0
run6	68	28	0	1	0	1	0
run3	13	6	0	0	0	0	10
run2	12	1	0	0	0	0	0
copy1	2	3	0	0	0	0	0
copy2	2	1	0	0	0	0	0
copy3	2	2	0	0	0	0	0
copy4	2	3	0	0	0	0	0
copy5	13	11	0	5	0	3	3
copy6	5	0	0	0	0	0	0

Table 11 compares the prespecified construction rules on ordered prefixes of the same campaign. The complete 20,000-run construction was collected regardless of any simulated earlier stopping decision, so the heuristic prefix counts are potential construction allocations, not executions actually avoided during this campaign. Validation and port outcomes are shared across rows. The singleton rule does not trigger on five inputs; their full prefixes and subsequent discoveries are still included. The no-discovery rule triggers on all inputs but leaves more held-out and port outside draws. No oracle is enlarged using those later discoveries, and no superiority claim is based on comparing these unequal budgets.

Table 11: Shared validation and port observations under ordered-prefix construction. Fired counts triggering inputs (out of 19); untriggered rules retain 20,000 runs. Construction budgets differ.

Construction rule	Build runs	Ref. new	Port new	Fired
Fixed 1,000/input	19,000	991	49	—
Fixed 5,000/input	95,000	383	33	—
Fixed 10,000/input	190,000	229	16	—
Fixed 20,000/input	380,000	124	8	—
Singleton heuristic	268,500	135	9	14
No-discovery heuristic	81,500	346	34	19

D Episodic Comparison Details

Construction-limited problems. Table 12 covers all five problems that did not freeze both populations by 2,000 episodes. Each frequency vector uses one earliest selected winner per answered complete episode. Reference vectors include the entire construction sample, even if one population froze earlier; they exclude validation. For `run4`, one engine-error episode leaves 1,999 reference winners per population. The other reference denominators are 2,000, and all ten port denominators are 100.

For an empirical reference vector $\hat{p}_R$ and port vector $\hat{p}_P$, the descriptive distance is $\widehat{\text{TV}} = \frac{1}{2} \sum_a |\hat{p}_R(a) - \hat{p}_P(a)|$, using the union of observed answers. The largest-gap answer maximizes the absolute share difference, with lexicographic ties. These descriptive summaries are uncorrected for sampling noise or adaptive construction: no p-values or acceptance cutoffs apply, and small distances do not establish equivalence. The accompanying saved-data audit retains all count vectors.

Table 12: Descriptive frequency comparisons for all five construction-limited inputs, using complete construction samples and 100 port episodes. Both native populations are shown; shares and empirical TV are rounded to four decimals. No coverage guarantees apply.

Input	Set	N_R	TV	Largest-gap answer	Ref. share	Port share
<code>misc4</code>	A	2,000	0.1995	<code>b</code>	0.0005	0.2000
<code>misc4</code>	B	2,000	0.0135	<code>y</code>	0.9865	1.0000
<code>run1</code>	A	2,000	0.0305	<code>mrrjjjj</code>	0.8070	0.8300
<code>run1</code>	B	2,000	0.0435	<code>mrrjjjj</code>	0.6935	0.7300
<code>run4</code>	A	1,999	0.0280	<code>yyz</code>	0.4087	0.3900
<code>run4</code>	B	1,999	0.0631	<code>xyd</code>	0.1431	0.0800
<code>fig5.4-top</code>	A	2,000	0.0195	<code>qeeq</code>	0.0535	0.0400
<code>fig5.4-top</code>	B	2,000	0.0405	<code>ixxq</code>	0.0390	0.0700
<code>eqe-baaab</code>	A	2,000	0.0060	<code>abbba</code>	0.0540	0.0600
<code>eqe-baaab</code>	B	2,000	0.0955	<code>qeeeq</code>	0.2755	0.2100

Episode and run denominators. Construction comprises 10,073 answered complete episodes and one error episode, with no entirely answerless complete episodes. Validation has 13,995 answered and five entirely answerless complete episodes; the port has 1,896 answered and four entirely answerless complete episodes. All nine entirely answerless episodes concern `copy5`. They remain in the phase totals but supply no selected answer. The two answer populations are not pooled as independent repetitions of these episodes.

Table 13: Episodic inner-run counts, including the inherited pilot. The four disjoint outcome columns sum to Attempted; No answer excludes caps and errors.

Phase	Attempted	Answered	No answer	Capped	Error
Construction	80,591	65,108	13,467	2,015	1
Validation	112,000	73,383	20,132	18,485	0
Port	15,200	9,815	3,388	1,997	0

The inherited `run4` construction error occurs at zero-based episode 46, first seed 90100368. It attempts seven runs: four answer, two end without an answer, and the seventh raises an unrecognized `get-bond-facet` message. The eighth run is not attempted. The four earlier answers remain in inner-run accounting, but the incomplete episode contributes no A or B winner. This explains the 207,791 actual runs rather than $25,974 \times 8 = 207,792$ scheduled runs. No error was retried as a replacement scientific observation.

Caps occur in 1,489 construction, 7,405 validation, and 863 port episodes, distinct from the capped inner-run counts in Table 13. Such episodes can still have a winner. Conditional winner coverage does not bound how often the system completes or answers an episode.

E Notation

Table 14: Notation distinguishes observed sets, unknown probabilities, and estimates.

Symbol	Meaning
p_R, p_P	Reference and port outcome laws for a fixed protocol
N_x, n_x	Reference and check sample sizes in runs or whole episodes, as specified
K, C_R	Number of check cycles and one-time reference cost
$C_{P,k}, C_{F,k}$	Check and follow-up costs for cycle k
L, ϕ	Episode horizon and fixed episode-outcome projection
J	Number of benchmark inputs, 19 in the case study
$c_x(o), f_{1,x}$	Reference outcome count and number of singleton outcomes
S_x, H_x, T_x	Observed reference set, selected empirical head, observed check set
$M_{R,x}, \widehat{M}_{R,x}$	Reference missing mass and its singleton-ratio estimate
$q_{P,x}$	Port probability of an outcome outside the frozen S_x
D_x, U_x	Number of novel draws and number of distinct novel outcomes

F Benchmark Inputs

Table 15 lists the complete benchmark. Up to three example answers per input are shared between the archived reference and post-repair CPU check, in decreasing reference-count order with lexicographic ties. They are observed outputs, not ground-truth labels.

Table 15: Benchmark inputs and observed shared answers; examples are not correctness labels.

Input	Initial	Modified	Target	Shared examples
misc4	a	b	z	b, z, y
fig5.7	aabc	aabd	ijkk	ijll, ijkl, ijl
misc3	abc	aabbcc	kkjjii	kkjjii, kkkjjjiii, kji
misc5	abc	abd	glz	gld, hlz, glz
misc2	abc	abd	ijk	ijl, ijd, ijk
run1	abc	abd	mrrjjj	mrrkkk, mrrjjk, mrrjjj
run4	abc	abd	xyz	xyd, wyz, xyz
misc1	abc	cba	mrrjjj	jjjrrm, mrrjjj, jrrmmm
fig5.4-top	eeqee	qeeq	xxixx	qeeq, ixxi, ixqx
eqe-baaab	eqe	qeq	abbba	baaab, qeeeq, qbbbq
run6	eqe	qeq	abbbc	qeeeq, baaaq, qcccb
run3	rst	rsu	xyz	xyu, wyz, xyz
run2	xqc	xqd	mrrjjj	mrrkkk, mrrjjk, mrrjjj
copy1	ab	c	ab	c, ab
copy2	bc	d	bc	d
copy3	xy	z	xy	z
copy4	zy	x	zy	x, zy
copy5	aabb	cc	aabb	cc, cac
copy6	abc	d	abc	d, abc